SQLite3 in Python (Complete Guide)

What is SQLite?

SQLite is a lightweight, serverless database that comes built into Python. You don't need to install any database server.

Python provides the **`sqlite3`** module to work with SQLite databases.

Advantages

- No installation required
- Fast and lightweight
- Stores data in a single `.db` file
- Easy to use
- Perfect for small applications, learning, and testing

Import sqlite3

```
import sqlite3
```

Create or Connect to a Database

If the database doesn't exist, it is created automatically.

```
import sqlite3

conn = sqlite3.connect("student.db")
```

Explanation

- `connect()` → Connects to a database
- `"student.db"` → Database file name
- If it doesn't exist, Python creates it.

Create Cursor

A cursor is used to execute SQL queries.

```
cursor = conn.cursor()
```

Think of Cursor as:

```
Python → Cursor → Database
```

Without a cursor, SQL commands cannot be executed.

Create Table

```
cursor.execute("""
CREATE TABLE students(
    id INTEGER PRIMARY KEY,
    name TEXT,
    age INTEGER,
    city TEXT
)
""")
```

Data Types

SQLite Type	Meaning
INTEGER	Whole Numbers
REAL	Decimal Numbers
TEXT	String
BLOB	Binary Data
NULL	Empty Value

Save Changes

```
conn.commit()
```

Without `commit()`, changes are not permanently saved.

Close Database

```
conn.close()
```

Always close the connection after work is completed.

Complete Example

```
import sqlite3

conn = sqlite3.connect("student.db")

cursor = conn.cursor()

cursor.execute("""
CREATE TABLE students(
id INTEGER PRIMARY KEY,
name TEXT,
age INTEGER,
city TEXT
)
""")

conn.commit()

conn.close()
```

Insert Data

Insert One Record

```
import sqlite3

conn = sqlite3.connect("student.db")
cursor = conn.cursor()

cursor.execute(
    "INSERT INTO students(name, age, city) VALUES (?, ?, ?)",
    ("Krish", 20, "Rajkot")
)

conn.commit()
conn.close()
```

Why `?` is used?

Instead of writing:

```
cursor.execute(
    "INSERT INTO students VALUES ('Krish',20,'Rajkot')"
)
```

Use:

```
VALUES (?, ?, ?)
```

This is **parameterized query**.

Benefits:

- Prevents SQL Injection

- Safer
- Cleaner

Insert Multiple Records

```
students = [
    ("Rahul",21,"Surat"),
    ("Amit",22,"Ahmedabad"),
    ("Neha",19,"Vadodara")
]

cursor.executemany(
    "INSERT INTO students(name, age, city) VALUES (?, ?, ?)",
    students
)

conn.commit()
```

Read Data

Fetch All Records

```
cursor.execute("SELECT * FROM students")

rows = cursor.fetchall()

for row in rows:
    print(row)
```

Output

```
(1, 'Krish', 20, 'Rajkot')
(2, 'Rahul', 21, 'Surat')
(3, 'Amit', 22, 'Ahmedabad')
```

Fetch One Record

```
cursor.execute("SELECT * FROM students")

row = cursor.fetchone()

print(row)
```

Output

```
(1, 'Krish', 20, 'Rajkot')
```

Fetch Limited Records

```
cursor.execute("SELECT * FROM students")

rows = cursor.fetchmany(2)

print(rows)
```

Output

```
[(1,'Krish',20,'Rajkot'),
 (2,'Rahul',21,'Surat')]
```

WHERE Clause

```
cursor.execute(
    "SELECT * FROM students WHERE city=?",
    ("Rajkot",)
)

print(cursor.fetchall())
```

ORDER BY

Ascending

```
cursor.execute(
    "SELECT * FROM students ORDER BY age ASC"
)
```

Descending

```
cursor.execute(
    "SELECT * FROM students ORDER BY age DESC"
)
```

UPDATE Data

```
cursor.execute(
    "UPDATE students SET city=? WHERE id=?",
    ("Junagadh",1)
)

conn.commit()
```

DELETE Data

```
cursor.execute(
    "DELETE FROM students WHERE id=?",
    (2,)
)

conn.commit()
```

DROP TABLE

```
cursor.execute("DROP TABLE students")
```

Deletes the entire table permanently.

Search Records

```
name = "Krish"

cursor.execute(
    "SELECT * FROM students WHERE name=?",
    (name,)
)

print(cursor.fetchall())
```

Count Records

```
cursor.execute(
    "SELECT COUNT(*) FROM students"
)

print(cursor.fetchone()[0])
```

Aggregate Functions

Average Age

```
cursor.execute(
    "SELECT AVG(age) FROM students"
)

print(cursor.fetchone()[0])
```

Maximum Age

```
cursor.execute(
    "SELECT MAX(age) FROM students"
```

```
)
```

Minimum Age

```
cursor.execute(
    "SELECT MIN(age) FROM students"
)
```

Sum

```
cursor.execute(
    "SELECT SUM(age) FROM students"
)
```

LIKE Operator

Starts with K

```
cursor.execute(
    "SELECT * FROM students WHERE name LIKE 'K%'"
)
```

Ends with h

```
cursor.execute(
    "SELECT * FROM students WHERE name LIKE '%h'"
)
```

Contains "ri"

```
cursor.execute(
    "SELECT * FROM students WHERE name LIKE '%ri%'"
)
```

LIMIT

```
cursor.execute(
    "SELECT * FROM students LIMIT 3"
)
```

Create Another Table

```
cursor.execute("""
CREATE TABLE courses(
course_id INTEGER PRIMARY KEY,
course_name TEXT
)
""")
```

INNER JOIN

```
cursor.execute("""
SELECT students.name, courses.course_name
FROM students
INNER JOIN courses
ON students.id = courses.course_id
""")
```

Transaction

```
try:
    cursor.execute(...)
    cursor.execute(...)

    conn.commit()

except:

    conn.rollback()
```

If an error occurs, all changes are rolled back.

Context Manager (`with` Statement)

```
import sqlite3

with sqlite3.connect("student.db") as conn:
    cursor = conn.cursor()

    cursor.execute(
        "SELECT * FROM students"
    )

    print(cursor.fetchall())
```

The connection is automatically committed (if successful) and closed when the `with` block ends.

Useful Cursor Methods

Method	Purpose
`execute()`	Execute one SQL statement

Method	Purpose
`executemany()`	Execute the same statement for multiple rows
`fetchone()`	Get the next row
`fetchmany(n)`	Get the next `n` rows
`fetchall()`	Get all remaining rows

Useful Connection Methods

Method	Purpose
`connect()`	Open or create a database
`commit()`	Save changes
`rollback()`	Undo uncommitted changes
`close()`	Close the connection
`cursor()`	Create a cursor

Complete CRUD Example

```
import sqlite3

# Connect
conn = sqlite3.connect("student.db")
cursor = conn.cursor()

# Create Table
cursor.execute("""
CREATE TABLE IF NOT EXISTS students(
    id INTEGER PRIMARY KEY,
    name TEXT,
    age INTEGER,
    city TEXT
)
""")

# Insert
cursor.execute(
    "INSERT INTO students(name, age, city) VALUES (?, ?, ?)",
    ("Krish", 20, "Rajkot")
)

# Read
cursor.execute("SELECT * FROM students")
print(cursor.fetchall())

# Update
cursor.execute(
    "UPDATE students SET city=? WHERE name=?",
    ("Ahmedabad", "Krish")
)
```

```
# Delete
cursor.execute(
    "DELETE FROM students WHERE name=?",
    ("Krish",)
)

# Save
conn.commit()

# Close
conn.close()
```

Interview Questions

1. What is SQLite?
2. What is the `sqlite3` module in Python?
3. What is the purpose of `cursor()`?
4. What is the difference between `fetchone()`, `fetchmany()`, and `fetchall()`?
5. Why is `commit()` required?
6. What happens if `commit()` is not called?
7. What is the purpose of `rollback()`?
8. Why should parameterized queries (`?`) be used instead of string formatting?
9. What is the purpose of `executemany()`?
10. What are the SQLite data types?

Link : CHATGPT

Link : ALL